

Description Of Your Assignment:

As you've learned in this class, every software project always starts in the same place: requirements. However, when you enter the real world you'll quickly discover that requirements don't show up like a nicely packaged homework assignment. The purpose of this assignment is to provide you with an opportunity to take real-world requirements created by real customers and transform them into software requirements that developers could use to create software.

This task starts off with you being provided with a transcript of a kickoff & requirements meeting that has happened between four members of the EventHub Corporation's ticketing application team and their launch customer. The people who participated in this call were:

1. Adaeze Okafor (Host) – Product Director for EventHub Ticketing
2. Marcus Bell (PM) – Product Manager
3. Thuy Nguyen – Quality Assurance (QA) Representative
4. Kenji Tanaka (Sec) – Cybersecurity Expert
5. Darnell Washington – Launch Customer / Student Entertainment Director at USF (managing 26,000 students)

What You Need To Do:

You need to read the transcript of the Kickoff meeting that has been provided to you. Having done this, you will want to present this transcript to a GenAI tool and request that it provide you with requirements that could be implemented by software engineers in order to create the ticketing application that EventHub wants to create.

It turns out that what the GenAI tool provides you with will NOT be perfect. Once you have the AI generated software requirements, you will need to carefully review them. Keeping in mind that the original customer requirements were created by real people, there are real possibilities that not all scenarios were covered. What you will be looking for will be issues with the generated requirements that have to do with:

1. Hallucinated Requirements - AI invents requirements that sound reasonable but were never requested.

2. Missing Requirements - AI focuses on the happy path.

3. Ambiguous Language - AI writes The report shall be generated quickly. How fast is "quickly"?

4. Mixing Functional and Nonfunctional Requirements - The application shall be easy to use.

5. Incorrect Business Logic - Platinum customers receive a 15% discount. / Premium customers receive a 15% discount

6. Inconsistent Terminology - Customer uses Customer, Order, Reservation AI later changes them to Client, Purchase, Booking



7. Missing Business Rules - Employees over 65 receive additional benefits. AI never identifies this as a business rule.

8. Missing Exceptions - AI forgets invalid password, locked account, expired password, MFA failure

9. Poor Requirement Granularity - The system shall manage users.

10. Non-Testable Requirements - The interface shall be user friendly.

11. Requirements that conflict with each other – Screens will be blue. Screens will be red.

If when reviewing your requirements you encounter a situation when there is an unanswered question (lets call them *irregular requirements*) that you need an answer to in order to create software requirement, you will reach out to the TA and ask your question.

Unlocking Mechanism

Since there are over 35 teams and a significant number of issues each of the teams can ask, we will be using the unlocking mechanism for irregular requirements. When any team asks about an irregular requirement we have planted in the transcripts, the TA will make a class-wide announcement providing further details about that irregular requirement.

End Result:

You will turn in the following items:

1. Your initial GenAI generated software requirements. [20% of grade]
2. Your list of issues that you found with your GenAI requirements. What did you find, why was it wrong, how did you fix it? [60% of your grade]
3. An updated and corrected version of the software requirements that would meet the needs of the EventHub team, in EARS format. [20% of your grade]



Call 1: Requirements Specification & Elaboration

Adaeze [Director]: Alright everyone, let's get into it. Goal for today is to nail down what EventHub actually does for version one. Framing first: EventHub is a public platform. Anyone in the world can register and buy tickets. Darnell [USF/Customer] is our launch customer and his campus is where we prove it works, but the requirements should hold for a global audience. Darnell [USF/Customer], start us off. What do your students need?

Darnell [USF/Customer]: Simple version: I manage entertainment for about 26,000 students at USF. Right now ticket sales are a mess: Google Forms, cash boxes, spreadsheets. I need one place where a student can see what's coming up, buy a ticket, and be done in under two minutes.

Marcus [Product Manager]: So the basic flow is browse, select, pay, confirm. That's the skeleton of our first version.

Darnell [USF/Customer]: Right. And students should be able to search. If they know Hozier is coming, they type Hozier, they find it.

Marcus [Product Manager]: Search by artist. What about by venue or date?

Darnell [USF/Customer]: Venue too. We run events at the Yuengling Center, the Ritz downtown, a couple of smaller rooms.

Adaeze [Director]: Noted. Browse and search by artist and venue. Accounts?

Marcus [Product Manager]: Yes, registration. Name, email, password. We need accounts for order history, since students lose paper tickets constantly. And one buying-experience idea from my notes: students are impatient, so on the event list, a single click opens the event page, but a quick double-click on the event card should just instantly buy one ticket at the student's saved settings. One gesture and they're done.

Darnell [USF/Customer]: Oh, they'd love that. They double-tap everything anyway.

Marcus [Product Manager]: Exactly. Frictionless.

Darnell [USF/Customer]: That's another thing. When they buy, they need to see their confirmation. And send them a confirmation email too, so they have it in their inbox.

Marcus [Product Manager]: Hmm. For version one we planned on-screen confirmation that they can re-open from order history. Sending email is a lot of extra work: making sure it gets delivered, avoiding spam filters, designing templates.

Darnell [USF/Customer]: Students live in their inbox. But okay, we can come back to this later.

Adaeze [Director]: Adding it to the follow-up list. Kenji [Security], jump in whenever. Thuy [QA], you too.

Kenji [Security]: I'm listening. Registration means passwords, so we'll get to that.

Thuy [QA]: Same. I want acceptance criteria on everything before we're done today.

Adaeze [Director]: Darnell [USF/Customer], tell them about the discount thing you raised with me.



Darnell [USF/Customer]: Yes. Big one. USF students get a discount. The way we detect them: if you register with an @usf.edu email address, you get the student price. There's no ID upload and no verification service. The email domain is the check.

Marcus [Product Manager]: Clean. I like it.

Kenji [Security]: Do we verify the email is real? Confirmation link?

Darnell [USF/Customer]: I mean… if they typed @usf.edu, that's good enough for me? Or maybe click a link. Whatever's standard.

Kenji [Security]: "Whatever's standard" isn't a requirement. Let's write that down as an open question.

Adaeze [Director]: Written down. How much is the discount, Darnell [USF/Customer]?

Darnell [USF/Customer]: We settled on a number internally, it's in the budget deck. I'll dig it up.

Adaeze [Director]: Please do. We can't build a discount without the number.

Marcus [Product Manager]: While we're on it, the discount applies to what, exactly? My assumption is only events that USF hosts. Your events, your discount.

Darnell [USF/Customer]: No no. Every event on the platform. That's the pitch I made to my students: you're USF, you save money on EventHub, period. If the Ritz lists a show, my students get the discount there too.

Marcus [Product Manager]: Darnell [USF/Customer], the platform is global. You're asking every organizer in every country to fund a discount for one Florida university's email domain. The Ritz isn't going to fund a discount they never agreed to, let alone a venue in Berlin.

Darnell [USF/Customer]: That's a business problem, not a student problem.

Adaeze [Director]: This is a real disagreement, and we won't settle it in this meeting. Marcus [Product Manager] and Darnell [USF/Customer], please continue this over email and copy me. Next topic.

Marcus [Product Manager]: Currency. Anyone anywhere can register and buy tickets to any listed event, so buyers pick their preferred currency and we display and charge in it.

Darnell [USF/Customer]: Which helps me too. I get asked about this constantly. International students paying with cards from home banks.

Adaeze [Director]: Live exchange rates?

Marcus [Product Manager]: No. Static rates in a config file. We update them manually, maybe once a semester. It's a pretend payment step anyway in the first version, with no real payment company involved.

Thuy [QA]: Which currencies?

Marcus [Product Manager]: The usual ones. Dollar, euro, pound… whatever international students actually use. We'll finalize the list.



Thuy [QA]: I can't write test cases against "the usual ones."

Marcus [Product Manager]: Understood, I owe you a list.

Kenji [Security]: Related question that nobody's asked: if a USF student pays in yen, does the discount come off before we convert or after? And where does rounding happen?

Marcus [Product Manager]: …That's a good question.

Darnell [USF/Customer]: Does it matter? It's the same money.

Kenji [Security]: It is absolutely not the same money once you round. Yen doesn't have cents.

Marcus [Product Manager]: Anyway. Inventory. Darnell [USF/Customer], walk us through capacity.

Darnell [USF/Customer]: Every event has a fixed number of tickets. When they're gone, they're gone. The site needs to show real-time availability. If there are 12 tickets left, students see 12, live.

Marcus [Product Manager]: Define "live." On page refresh? Auto-updating?

Darnell [USF/Customer]: Live means live. When I look at my phone, the number is right.

Thuy [QA]: The number can be right when the page loads and outdated four seconds later. Those are different systems.

Darnell [USF/Customer]: You're the experts. It should just… be accurate.

Adaeze [Director]: We'll spec it precisely later. Related question: what happens between picking tickets and paying?

Darnell [USF/Customer]: The tickets should be held for you. Like, you pick four tickets, you have ten minutes to check out, nobody can grab them from under you.

Marcus [Product Manager]: I'd push back for the first version. Holds mean timers, expiry logic, released inventory, all sorts of edge cases. Simpler rule: nothing is yours until you complete the purchase. If someone beats you to the last ticket, you get a friendly error.

Darnell [USF/Customer]: My students will riot if they lose tickets mid-checkout for a big show.

Marcus [Product Manager]: And we'll miss launch if we build a reservation engine.

Adaeze [Director]: That's the third item for the email discussions. Kenji [Security], your turn. Security requirements.

Kenji [Security]: Passwords are hashed and never stored in plaintext; that one is non-negotiable. Sessions expire. Checkout and order history require login. Quantity fields are validated on the server, so zeros, negatives, and absurd numbers are all rejected. Currency comes from our supported list only; never trust a currency string from the client.

Thuy [QA]: All testable. I want each of those as its own requirement line.

Kenji [Security]: And purchase limits. Something like max four tickets per account per event.



Darnell [USF/Customer]: Four? No. Student org presidents buy thirty at a time for their members. Greek life buys blocks of fifty. If you cap at four, my biggest buyers can't use the platform.

Kenji [Security]: Then your biggest buyers look exactly like bots, and I can't protect the inventory.

Darnell [USF/Customer]: So make an exception for orgs.

Kenji [Security]: Verified how? By whom?

Adaeze [Director]: That's four email discussions now. This is going to be a busy inbox. Thuy [QA], quality requirements.

Thuy [QA]: Every purchase attempt, success or failure, gets logged: timestamp, user, event, quantity, currency, outcome. That's my audit trail for debugging and for disputes.

Darnell [USF/Customer]: Disputes happen weekly, so yes please.

Thuy [QA]: And I want failures to be specific. "Sold out" is a different message than "invalid quantity" which is different from "unsupported currency."

Marcus [Product Manager]: Agreed, error messages are part of the spec. And to be precise on my earlier point, the quantity cap is a per-order thing. No single order above the limit.

Kenji [Security]: Per account, per event. That's what keeps a scalper from placing ten orders.

Marcus [Product Manager]: Sure, sure.

Darnell [USF/Customer]: Oh, one more thing while I remember. Can students pick their seats? Like a little map of the venue where you tap your seat?

Marcus [Product Manager]: That's something for a future version. Seat maps are an enormous feature.

Darnell [USF/Customer]: Fine, fine. What about an app? iPhone app? The web is fine but students are app people.

Marcus [Product Manager]: Also a future version. For now it's a website that adjusts to phone screens, and it'll work fine.

Darnell [USF/Customer]: You product people love the words "future version."

Adaeze [Director]: It's the most beautiful phrase in the language. Order history is the last item to be discussed in today's meeting. Marcus [Product Manager], walk us through it, please.

Marcus [Product Manager]: Logged-in users see all past orders. Each order re-opens as a confirmation-style page: event, quantity, currency paid, rate applied, total. That's also our answer to lost tickets.

Darnell [USF/Customer]: Works, as long as the door staff can trust what's on the screen. Actually, on that: make it so students can't screenshot the confirmation page. If screenshots float around a group chat, we'll have twenty kids showing the same ticket.

Marcus [Product Manager]: Noted. No-screenshot on the confirmation page.



Thuy [QA]: What are the door staff actually checking, though? Anyone can show a confirmation page twice.

Darnell [USF/Customer]: Today we just check names off a list…

Marcus [Product Manager]: Right. Anyway, I think we have our list.

Adaeze [Director]: Let's wrap. Marcus [Product Manager], summarize.

Marcus [Product Manager]: Summary: registration and login, browse and search events by artist and venue, event pages showing availability, a purchase flow with quantity and currency selection, fixed exchange rates, the USF email discount (details still missing), on-screen confirmation, order history, a log of every purchase attempt, checking all inputs on the server, and Kenji [Security]'s security requirements. Questions going to email: email versus on-screen confirmation, which events the discount covers, the discount amount, whether tickets are held during checkout, purchase limits, and the currency list.

Adaeze [Director]: Six open questions from one call. Sounds about right. Thanks everyone. Architecture call in about two weeks.